INTERVIEW PREP | BEGINNER

DSA in C++

Zero to Interview Ready

Arrays • Strings • Linked Lists • Stacks & Queues

Recursion • Sorting • Searching • Time Complexity

Beginner | C++17 | Free

01 Arrays & Vectors

02 Strings

03 Linked Lists

04 Stack & Queue

05 Recursion

06 Sorting

07 Binary Search

08 Hashing

09 Time Complexity

Curated for your first technical interview | Practice each example before moving on

TABLE OF CONTENTS

CHAPTER 01

Time & Space Complexity

Understand Big-O before anything else

Before writing a single line of DSA code, you must understand **Big-O notation**. Interviewers always ask 'what is the time complexity of your solution?'

Big-O Cheat Sheet

Notation	Name	Example
O(1)	Constant	Array index access
O(log n)	Logarithmic	Binary search
O(n)	Linear	Single loop
O(n log n)	Linearithmic	Merge sort, std::sort
O(n²)	Quadratic	Nested loops
O(2■)	Exponential	Brute-force subsets

■ **REMEMBER**

Always aim for O(n) or O(n log n). If you have O(n²), ask yourself: can I use a hash map or sorting to reduce it?

Rules of Thumb

- Drop constants: O(2n) → O(n)
- Drop lower-order terms: O(n² + n) → O(n²)
- Each nested loop multiplies: two nested loops over n → O(n²)
- Recursion tree depth × work per level = total complexity

CHAPTER 02

Arrays & Vectors

The most fundamental data structure

Arrays store elements in **contiguous memory**. In C++, prefer **std::vector** over raw arrays for flexibility.

Declaration & Basics

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Static array
    int arr[5] = {10, 20, 30, 40, 50};

    // Dynamic vector (use this in interviews!)
    vector<int> v = {1, 2, 3, 4, 5};
    v.push_back(6);          // add to end  O(1) amortised
    v.pop_back();            // remove last  O(1)
    cout << v.size();        // 5
    cout << v[2];            // 3  (0-indexed)

    // Traversal
    for (int x : v) cout << x << ' ';
    // or with index
    for (int i = 0; i < v.size(); i++) cout << v[i];
}
```

Must-Know Vector Operations

Operation	Code	Time
Access element	`v[i]  or  v.at(i)`	O(1)
Insert at end	`v.push_back(x)`	O(1)*
Remove from end	`v.pop_back()`	O(1)
Insert at position	`v.insert(v.begin()+i, x)`	O(n)
Size	`v.size()`	O(1)
Sort	`sort(v.begin(),v.end())`	O(n log n)
Reverse	`reverse(v.begin(),v.end())`	O(n)
Find	`find(v.begin(),v.end(),x)`	O(n)

Classic Interview Problem: Find Maximum

```cpp
// Find the maximum element in an array
int findMax(vector<int>& arr) {
    int maxVal = arr[0];
    for (int i = 1; i < arr.size(); i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }
    return maxVal;   // Time: O(n)  Space: O(1)
}
```

> ✔ **TIP**
> Always clarify: is the array sorted? Can there be duplicates? Can values be negative? These details can change your approach.

CHAPTER 03

Strings

Character arrays wrapped in a class

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s = "hello";
    cout << s.length();          // 5
    cout << s[0];                // 'h'
    s += " world";              // concatenation
    cout << s.substr(0, 5);      // "hello"
    cout << s.find("world");    // index of 'w' = 6

    // Reverse a string
    reverse(s.begin(), s.end());

    // Convert char to int and back
    char c = '7';
    int n = c - '0';             // n = 7
    char back = '0' + n;         // back = '7'
}
```

Classic: Check if Palindrome

```cpp
bool isPalindrome(string s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] != s[right]) return false;
        left++;  right--;
    }
    return true;   // Time: O(n)  Space: O(1)
}
```

Useful String Functions

- **s.size() / s.length()** — number of characters
- **s.substr(start, len)** — extract substring
- **s.find(t)** — position of t in s (string::npos if not found)
- **stoi(s)** — string to int | **to_string(n)** — int to string
- **isalpha(c), isdigit(c), tolower(c), toupper(c)** — char helpers

NOTE

Strings in C++ are mutable, unlike Java/Python. s[i] = 'x' is perfectly valid and runs in O(1).

CHAPTER 04

Linked Lists

Dynamic data, pointer magic

A linked list is a chain of **nodes**, each holding a value and a pointer to the next node. Unlike arrays, insertions/deletions at the head are O(1).

Node Structure & Basic Operations

```
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Insert at head   O(1)
void insertHead(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;
    head = newNode;
}

// Print list   O(n)
void printList(Node* head) {
    while (head != nullptr) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL";
}
```

Classic: Reverse a Linked List

```
Node* reverseList(Node* head) {
    Node *prev = nullptr, *curr = head, *nxt = nullptr;
    while (curr != nullptr) {
        nxt        = curr->next;   // save next
        curr->next = prev;         // reverse link
        prev       = curr;         // move prev forward
        curr       = nxt;          // move curr forward
    }
    return prev;   // new head  |  Time: O(n)  Space: O(1)
}
```

Classic: Detect a Cycle (Floyd's Algorithm)

```
bool hasCycle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;          // 1 step
        fast = fast->next->next;    // 2 steps
        if (slow == fast) return true;
    }
    return false;   // Time: O(n)  Space: O(1)
}
```

REMEMBER

Always check for nullptr before accessing ->next. Null pointer dereference is the #1 linked list bug!

CHAPTER 05

Stack

Last In, First Out (LIFO)

A stack follows **LIFO** order. Use STL **std::stack** in interviews. Think of it as a pile of plates — you add/remove only from the top.

```
#include <stack>
using namespace std;

stack<int> st;
st.push(10);       // push  O(1)
st.push(20);
st.push(30);
cout << st.top();  // 30  (peek)  O(1)
st.pop();          // removes 30  O(1)
cout << st.size(); // 2
cout << st.empty();// false
```

Classic: Valid Brackets

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c=='(' || c=='{' || c=='[') {
            st.push(c);
        } else {
            if (st.empty()) return false;
            char top = st.top(); st.pop();
            if (c==')' && top!='(') return false;
            if (c=='}' && top!='{') return false;
            if (c==']' && top!='[') return false;
        }
    }
    return st.empty();   // Time: O(n)  Space: O(n)
}
```

✔ TIP

Stack is the go-to structure for: bracket matching, undo/redo operations, DFS traversal, and next greater element problems.

CHAPTER 06

Queue

First In, First Out (FIFO)

```cpp
#include <queue>
using namespace std;

queue<int> q;
q.push(10);        // enqueue  O(1)
q.push(20);
q.push(30);
cout << q.front(); // 10  (peek front)  O(1)
cout << q.back();  // 30  (peek back)   O(1)
q.pop();           // removes 10  O(1)
cout << q.size();  // 2
```

Priority Queue (Max-Heap by default)

```cpp
priority_queue<int> pq;         // max-heap
pq.push(5); pq.push(1); pq.push(9);
cout << pq.top();               // 9  (largest first)

// Min-heap
priority_queue<int, vector<int>, greater<int>> minPQ;
minPQ.push(5); minPQ.push(1); minPQ.push(9);
cout << minPQ.top();            // 1  (smallest first)
```

NOTE

Queue is essential for BFS (Breadth-First Search). Priority queues are used in Dijkstra's shortest path and top-K problems.

CHAPTER 07

Recursion

Solving big problems by solving smaller ones

Every recursive function needs a **base case** (to stop) and a **recursive case** (to make progress). Without a base case, you get infinite recursion (stack overflow).

Factorial & Fibonacci

```
// Factorial: n! = n * (n-1)!
int factorial(int n) {
    if (n <= 1) return 1;       // base case
    return n * factorial(n-1);  // recursive case
}
// factorial(5) = 5*4*3*2*1 = 120  |  Time: O(n)

// Fibonacci (naive - O(2^n), avoid in interviews)
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

// Fibonacci with memoisation - O(n)
int memo[1001];
int fibMemo(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}
```

The 3-Step Recursion Template

- **Step 1 — Base case:** What is the simplest input? Return immediately.
- **Step 2 — Shrink:** Call the function with a smaller input.
- **Step 3 — Combine:** Use the result of the smaller call to build the answer.

■ REMEMBER

Naive recursion for Fibonacci is O(2■). Always mention memoisation (top-down DP) when you see overlapping sub-problems.

CHAPTER 08

Sorting Algorithms

Know the concepts, use STL sort in practice

Algorithm	Best	Avg	Worst	Space	Stable
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	O(n lg n)	O(n lg n)	O(n lg n)	O(n)	Yes
Quick Sort	O(n lg n)	O(n lg n)	O(n²)	O(lg n)	No
std::sort	O(n lg n)	O(n lg n)	O(n lg n)	O(lg n)	No*

Using std::sort (Always use this in interviews)

```
vector<int> v = {5, 2, 8, 1, 9};

sort(v.begin(), v.end());            // ascending
sort(v.begin(), v.end(), greater<int>()); // descending

// Custom comparator (sort by second element of pair)
vector<pair<int,int>> pairs = {{1,3},{2,1},{3,2}};
sort(pairs.begin(), pairs.end(), [](auto& a, auto& b){
    return a.second < b.second;
});
```

Insertion Sort (Understand the logic)

```
void insertionSort(vector<int>& arr) {
    for (int i = 1; i < arr.size(); i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

✔ TIP

In an interview: always use std::sort (O(n log n)). Mention Merge Sort when asked to implement sorting from scratch.

CHAPTER 09

Binary Search

Halve the search space every step

Binary search works on **sorted arrays**. Each iteration eliminates half the remaining elements → O(log n).

Classic Binary Search Template

```
// Returns index of target, or -1 if not found
int binarySearch(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // avoids overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left  = mid + 1;
        else                        right = mid - 1;
    }
    return -1;   // Time: O(log n)  Space: O(1)
}
```

STL Binary Search Helpers

```
vector<int> v = {1, 3, 5, 7, 9};

// Check existence
bool found = binary_search(v.begin(), v.end(), 5);  // true

// First position >= target
auto it = lower_bound(v.begin(), v.end(), 5);
int idx = it - v.begin();   // idx = 2

// First position > target
auto it2 = upper_bound(v.begin(), v.end(), 5);
int idx2 = it2 - v.begin(); // idx2 = 3
```

■ REMEMBER

Use mid = left + (right - left) / 2 — NOT (left + right) / 2. The latter overflows when left + right > INT_MAX.

CHAPTER 10

Hashing — Maps & Sets

O(1) average lookup, insertion, deletion

```cpp
#include <unordered_map>
#include <unordered_set>
using namespace std;

// unordered_map: key -> value
unordered_map<string, int> freq;
freq["apple"] = 3;
freq["banana"]++;
if (freq.count("apple")) cout << "found";
for (auto& [key, val] : freq) cout << key << ':' << val;

// unordered_set: unique keys
unordered_set<int> seen;
seen.insert(42);
if (seen.count(42)) cout << "already seen";
```

Classic: Two Sum

```cpp
// Given array + target, return indices of two numbers that sum to target
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen;  // value -> index
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement))
            return {seen[complement], i};
        seen[nums[i]] = i;
    }
    return {};   // Time: O(n)  Space: O(n)
}
```

✔ TIP

Whenever you see O(n²) from nested loops checking for pairs, think: can a hash map reduce this to O(n)?

Ordered vs Unordered

	unordered_map/set	map/set (ordered)
Lookup	O(1) avg	O(log n)
Insert	O(1) avg	O(log n)
Sorted order?	No	Yes (ascending)
Use when	Need speed	Need sorted keys

CHAPTER 11

Two Pointers & Sliding Window

Core interview patterns — recognise them fast

Two Pointers — Pair Sum in Sorted Array

```cpp
// Find pair with given sum in SORTED array
bool pairSum(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return true;
        else if (sum < target) left++;
        else                   right--;
    }
    return false;   // Time: O(n)  Space: O(1)
}
```

Sliding Window — Max Sum Subarray of Size K

```cpp
int maxSumSubarray(vector<int>& arr, int k) {
    int windowSum = 0;
    // Build first window
    for (int i = 0; i < k; i++) windowSum += arr[i];
    int maxSum = windowSum;
    // Slide the window
    for (int i = k; i < arr.size(); i++) {
        windowSum += arr[i] - arr[i-k]; // add new, remove old
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;   // Time: O(n)  Space: O(1)
}
```

When to Use Each Pattern

Pattern	When to use	Key signal
Two Pointers	Sorted array, pair/triplet sums, palindrome	Sorted + pairs
Sliding Window	Subarray/substring with constraint	Contiguous range
Hash Map	Unsorted, fast lookup, counting freq	Pairs in O(n)
Binary Search	Sorted data or 'find minimum possible'	log n needed
Stack	Matching brackets, next greater element	LIFO pattern

CHAPTER 12

Quick Reference Cheat Sheet

Everything on one page — print and stick it up

C++ STL Quick Map

Container	Header	Key Operations
vector<T>	<vector>	push_back, pop_back, size, [], sort, reverse
string	<string>	length, substr, find, +=, stoi, to_string
stack<T>	<stack>	push, pop, top, empty, size
queue<T>	<queue>	push, pop, front, back, empty
priority_queue	<queue>	push, pop, top (max-heap default)
unordered_map	<unordered_map>	[], count, find, erase, size
unordered_set	<unordered_set>	insert, count, find, erase
pair<A,B>	<utility>	.first, .second, make_pair(a,b)

Complexity Quick Recall

```
Array access      O(1)     |  Linked list access    O(n)
Array search      O(n)     |  Binary search         O(log n)
Hash map lookup   O(1)avg  |  BST lookup            O(log n)
Stack push/pop    O(1)     |  Queue push/pop        O(1)
Bubble/Insert sort O(n^2)  |  Merge/Quick sort      O(n log n)
std::sort         O(n log n)|  std::binary_search   O(log n)
```

The Interview Checklist

- Clarify the problem — repeat it back, ask about edge cases
- Think out loud — discuss brute force first, then optimise
- Write the function signature before filling in the body
- Handle edge cases: empty array, single element, all same
- State time and space complexity BEFORE they ask
- Test with a small example on paper before coding
- Know: vector, string, map, set, stack, queue from STL
- Practise: Two Sum, Reverse LL, Binary Search, Valid Brackets

NOTE

Start with LeetCode Easy problems once you finish this guide. Recommended first 10: Two Sum, Reverse String, Valid Parentheses, Merge Sorted Array, Climbing Stairs, Best Time to Buy Stock, Maximum Subarray, Contains Duplicate, Move Zeroes, Linked List Cycle.